

ResIR Compiler Report

CPS2000 – Compiler Theory and Practice

Name: **Dave Galea**

Student ID: **0283404L**

Implementation Language:  **Rust**

June 2026

Abstract

While a report cannot fully capture all parts of this project without becoming a novel, it can provide an insight to most of the important design decisions taken throughout this assignment.

This report is therefore my best attempt at expressing the work done for the fulfillment of the course CPS2000: Compiler Theory and Practice. It has discussion regarding implementation, testing, and any specifics related to a particular piece of the Compiler. Any limitations and ideas for future improvement are also included toward the end of the report.

The link for the explanation video can be found here: <https://drive.google.com/file/d/1rIBUSCF6zFFP6oiAW43uhfWXDCihb0Wh/view?usp=sharing>

Contents

1	Project Overview	3
1.1	Aim	3
1.2	Pipeline	3
1.3	Repository Layout	3
1.4	Running and Testing the Compiler	4
2	Lexical Analysis	5
2.1	Task Overview	5
2.2	Design	5
2.3	Main Functions	7
2.4	Error Handling	7
2.5	Testing	7
3	Syntax Analysis	8
3.1	Task Overview	8
3.1.1	Internal Representation	8
3.2	Design	8
3.3	Error Handling	10
3.4	Testing	10
4	Semantic Analysis	11
4.1	Task Overview	11
4.2	Checks Implemented	11
4.3	Design	11
4.4	Error Handling	13
4.5	Testing	13
5	Code Generation	14
5.1	Task Overview	14
5.2	Design	14
5.3	Main Functions	15
5.4	Type Mapping	15
5.5	Operation Lowering	15
5.6	Testing	16
6	Program Analysis with Ascent	17
6.1	Task Overview	17
6.2	Design	17
6.3	Testing	18
7	Concluding Notes	19
7.1	Limitations	19
7.2	Future Improvement	19
7.3	A Note on Rust for Compiler Construction	19
7.4	Deviations from the Supplied grammar	19

A Sample ResIR Inputs	21
A.1 Assignment Example 1	21
A.2 Assignment Example 2	21
A.3 Assignment Example 3	22
B Generated C Outputs	23
B.1 Assignment Example 1	23
B.2 Assignment Example 2	23
B.3 Assignment Example 3	24
C Full Test Harness Output	25

1 Project Overview

1.1 Aim

The goal of this project is to build a compiler pipeline for an Intermediate Representation (IR) called **ResIR**, through which **ResIR** source code is inputted, and **Compilable C** is outputted.

1.2 Pipeline

The compiler pipeline structured as follows:

Stage	Input	Output
Lexical analysis	ResIR source code	Tokens
Syntax analysis	Tokens	Filled Intermediate Representation + CFG
Semantic analysis	Program + CFG	Result of the Check
Code generation	Program and Ascent output	C source code with no unreachable blocks
Program analysis	CFG facts	Ascent output

1.3 Repository Layout

```
1  src/
2  |-- samples.rs --> holds all test examples
3  |-- test.rs --> test harness which runs the examples
4  |-- main.rs
5  |-- parser/
6  |   |-- cfg.rs
7  |   |-- parser_core.rs
8  |   |-- ir.rs
9  |   '-- mod.rs
10 |-- codegen/
11 |   |-- generator.rs
12 |   '-- mod.rs
13 |-- lexer/
14 |   |-- table.rs
15 |   |-- lexer_core.rs
16 |   |-- tokens.rs
17 |   '-- mod.rs
18 |-- semantic/
19 |   |-- analyser_core.rs
20 |   '-- mod.rs
21 '-- datalog/
22 |   |-- datalog_analyser.rs
23 |   '-- mod.rs
24
```

Credit for the above: <https://utilitycove.com/tools/folder-structure-generator>

1.4 Running and Testing the Compiler

Note

Regarding the header file; my understanding was that it will be provided by whoever will be testing the system. The program thus optionally takes as input the name of the header file, and **adapts to the name you provide**, so if your file is called `abcdefheader.h`, and you pass it as an argument, the generated C will contain `#include "abcdefheader.h"`

Limitation / known issue

The header argument is only optional for inputs that do not reference a custom type or external function. For the struct and call examples (such as examples 2 and 3), the header **must** be passed, otherwise the generated `#include` falls back to a default name and will not match the header the who ever is testing provides.

The program takes input as follows, with the header file being optional:

From the root directory:

```
cargo run -- <input.resir> <output.c> [header.h]
```

This then generates a compilable C file. A self-contained function compiles and links directly with::

```
gcc -std=c11 -Wall -Wextra -pedantic filename.c
```

A function that calls an external helper or uses a custom struct compiles as a translation unit, and is linked together with the file(s) defining those helpers and the provided header, for example:

```
gcc -std=c11 -Wall -Wextra -pedantic file1.c file2.c driver.c -o demo
```

Regarding testing, a simple `cargo test` runs the entire test harness, and produces an output specifying the tests and their outcome.

2 Lexical Analysis

2.1 Task Overview

The first step in the compilation process is here: The job of this portion of the compiler is to take any ResIR source program, and transform it into tokens.

The division of labour is thus split across three files:

- `tokens.rs`: Definition of token data structures.
- `table.rs`: DFSA states, character categories, any useful conversions (like from a state to a token), and obviously, the transition table.
- `lexer_core.rs`: the scanning loop that finally produces the tokens.

This stage was by far the longest stage to implement, even though the marks allocated were smaller than other parts like the parser and the code generation. This was because a deliberate effort was made to try and make it as well prepared to add on to as possible, as I knew that as the system grew more complex, a solid base to lie on was necessary.

2.2 Design

A table driven approach:

The lexer uses a table-driven Deterministic Finite State Automaton (DFSA)[Figure 2.1]. Having previously done a course which touched on compiler construction; the requirement for it being table was rather unintuitive, as I was more familiar with a 'direct-coded' approach using conditional logic. Regardless, a table-driven lexer helped me regarding separation of concerns of the different parts of the lexer, as well as give me a bit of a better insight on how the DFSA ought to function.

Maximal Munch

The concept of maximal munch (or simply longest match) was implemented for outputting tokens. This was done in such a way that when several prefixes of the remaining input could form valid tokens, the lexer chooses the longest one[2]. This implementation is necessary for tokens such as `:` and `::`, where the lexer should produce a path separator when two colons appear together, rather than producing two separate colon tokens.

Batch model

I got inspired by the Batch model from the incredible book *Crafting Interpreters*[1]. Although the lecture notes follow a stream model, where the parser receives one token at a time and parses by repeatedly requesting the next token from the lexer, I opted for a batch model. which produces an entire list of tokens in one go, after which the entire batch of tokens is parse

This choice was made on the basis that in my mind, having the lexer as a separate process which takes in a "bucket" of input and produces a "bucket" of tokens, makes more intuitive sense. This idea of "division of labour" It allowed me to reason more about how the lexer will work without having to reason about parser structure alongside it.



Figure 2.1: DFSA used by the table-driven lexer

2.3 Main Functions

Function / Structure	Role
<code>Token</code> , <code>TokenKind</code> , <code>TokenAttribute</code>	<code>TokenKind</code> stores the kind of token produced by the lexer, <code>TokenAttribute</code> stores the lexeme and source location, and <code>Token</code> combines both.
<code>build_transition_table()</code>	Builds the DFSA transition table as follows: <code>table = [state][category]</code> .
<code>produce_tokens(input)</code>	This is the main lexer function, used by the rest of the compiler, which after building the transition table, loops over the input while the end of a file is not detected. In this loop, the location of the current token is tracked, <code>next_token</code> is called repeatedly to read another lexeme. Each lexeme is checked to see whether it is a reserved keyword, or a label. If neither, it is classified as an Identifier. After the last token is read, an end of file token is appended and the full Vector of tokens is returned
<code>next_token(...)</code>	Scans one token: it walks the DFSA, then rolls back to the last accepting state and returns its kind and lexeme.

2.4 Error Handling

Lexical error handling concerns finding if a character is valid or not. If the lexer finds an invalid character, it returns as follows.

```
Lexical Error: Unexpected character ' ' at line ' ', col ' '
```

2.5 Testing

Testing Methodology

Lexical testing revolved around confirming each token class is recognised correctly, and tested for keyword/label distinctions by asserting (`input`, `expected token stream`) rows through a single loop, with a negative test for invalid input.

Test / Input	Expected outcome
Three assignment examples	No error; stream ends in <code>EndOfFile</code>
<code>%x = const 0</code>	<code>Local</code> , <code>Equals</code> , <code>Const</code> , <code>IntegerLiteral</code>
<code>function</code>	<code>Function</code> , not <code>Identifier</code>
<code>bb0 bb</code>	<code>Label</code> then <code>Identifier</code> (<code>bb</code> is not a label)
<code>::</code>	<code>::</code> lexes as one <code>PathSep</code> , not two <code>Colon</code>
<code>12.34</code>	single <code>FloatLiteral</code>
<code>invalid characters</code>	<code>Err</code> containing "Lexical Error"
<code>function\n @</code>	<code>Err</code> naming the correct line (2) and column (3)

3 Syntax Analysis

3.1 Task Overview

With a batch of tokens now available, the next step is to give them shape. The job of this portion of the compiler is to take the `Vec` of tokens produced by the lexer and build an in-memory representation of the program as well as a Control Flow Graph (CFG).

The division of labour is split across three files:

- `ir.rs`: the IR data structures of which every node of the program is built from (`Program`, `Function`, `Block`, `Stmt`, `Rhs`, `Term`, `Type`, and so on).
- `parser_core.rs`: the parser itself, with one `Parse_x` function per grammar rule.
- `cfg.rs`: builds a Control Flow Graph from the parsed program.

3.1.1 Internal Representation

The parser ultimately produces a `Program`. A program contains optional external type declarations and is limited to one function per program.

```
1      pub struct Program {
2          pub externtypes: Vec<ExternType>,
3          pub function: Function,
4      }
5
6      pub struct Function {
7          pub path: Path,
8          pub params: Option<Params>,
9          pub rettype: RetType,
10         pub locals: Vec<(Local, Type)>,
11         pub entry: Label,
12         pub blocks: Vec<Block>,
13     }
14
```

3.2 Design

A recursive descent parser:

This was another major debate on continuing with the implementation. In the previous course I followed, this time, the table-driven approach was the more familiar. We worked many examples by hand, so my familiarity drew me to it. However, as I quickly found out, knowing how to calculate something by hand does not mean it is trivial to implement it by hand, and a quick google search led me to find that table-driven parsing is one of the more difficult approaches to take. Thus, an alternative was needed.

Once again, I landed on Crafting Interpreters, which ultimately landed me on Recursive Descent. The idea is simple: For each grammar rule, the non terminal `x` gets parsed with a `parse_x`

function, and terminals, which follow the non-terminal, are checked to match the expected token; *the expected token is based on the grammar rule the non-terminal specifies*.

An example is as follows:

The grammar specifies `Term ::= 'jump' Label`

The parser proceeds to call `parse_term()`, for the non terminal 'Term'(Terminator). Then, checks that the following token matches the terminal: `jump`. After verifying that this is correct, it calls `parse_label()` for the final non terminal.

Navigation helpers

A small set of helper functions sits underneath the parsing rules: `peek()` to look without consuming, `advance()` to consume, `check_tokenkind()` to test the current token, `match_token()` to optionally consume one, and `consume_token()` to demand a specific token, and raise an error if it doesn't match. These were once again inspired from the book *Crafting Interpreters*.

Lookahead

As specified in the assignment, parsing requires only one token of lookahead in *almost* all cases. The one time that this is not applicable is for statements, where:

`Stmt ::= [Local '=' ] Rhs`

This is handled in `parse_stmt` by remembering the current position, peeking one token ahead to check for an `=`, and then restoring the position before actually parsing.

A separate CFG

The parser produces a plain IR from which the Control Flow Graph is then built separately in `cfg.rs` by `build_cfg`. Keeping CFG construction out of the parser meant the parser only had to worry about syntax, while later stages (especially Program Analysis) had a ready made graph to work over.

```

1      pub struct Cfg {
2          pub blocks: Vec<Block>,
3          pub edges: Vec<(Label, Label)>,
4          pub entry: Label,
5      }
6

```

Terminator	CFG edges
<code>return</code>	none
<code>jump bbX</code>	current block $\rightarrow$ bbX
<code>cjump %c, bbX, bbY</code>	current block $\rightarrow$ bbX, current block $\rightarrow$ bbY

3.3 Error Handling

Syntax error handling concerns finding broken grammar rules, through expectation of tokens at under specific circumstances. If the Parser finds a broken grammar rule, it produces a syntax error through `yell_error`, which returns as follows.

Syntax Error: <message> at Line: ' ', Col: ' '

For example:

Syntax Error: expected ; after entry label at Line: 4, Col: 5

3.4 Testing

Testing Methodology

Syntax testing revolved around confirming that the appropriate structs in the IR are filled adequately, and that any incorrect grammar rules do not allow the compiler to continue via a syntax error.

Test / Input	Expected outcome
Three assignment examples	Parse with no error
<code>%x = const 0</code>	<code>Rhs::Const</code> ; block ends in <code>Term::Return</code>
<code>bin add</code>	<code>Rhs::Bin</code>
<code>call Math::abs_diff</code>	<code>Rhs::Call</code>
<code>member_ptr %p, x</code>	<code>Rhs::Member_ptr</code>
<code>cjump %c, bb1, bb2</code>	<code>Term::CJump</code>
missing ;	Err (syntax error)
missing entry	Err (syntax error)

4 Semantic Analysis

4.1 Task Overview

With a parsed program and its CFG acquired, the next step is to derive *meaning* before any C is generated. It acts as a validation check that the program has to get through after parsing before being able to continue to the final stages of compilation.

Unlike the previous stages, this one lives almost entirely in a single file:

- `analyser_core.rs`: a small `SymbolTable`, and a sequence of independent check functions driven by one `analyse` function.

4.2 Checks Implemented

Check	Description
Declared locals	Every local used as a destination, an operand, or in a terminator must exist in the symbol table.
Unique labels	A block label must not appear more than once.
Entry exists	The declared entry block must exist among the blocks.
Targets exist	Every jump and cjump target must refer to a real block.
Operand types	operations must use compatible types; in general a statement's left- and right-hand types must be compatible or equal.
Return type	Every return must match the function's return type, and a void function must not return a value.
CJump condition	A cjump condition must have type <code>bool</code> .
Extern types	Every custom (path) type used by fields, parameters, the return type, locals, or a cast must have a matching extern type declaration.

4.3 Design

A symbol table:

Inspired from the lecture notes, The `SymbolTable` is simply a `HashMap` from a local's name to its `Type`, filled once from the parameters and then the locals; i.e. places where declaration of variables can take place. Every later check that needs a type simply looks it up here, and a failed lookup is itself the "undeclared local" error.

```
1      let mut symboltable = SymbolTable::new();
2      if let Some(params) = &function.params {
3          for param in &params.params {
4              symboltable.insert(&param.local.ident.string, param.typealt.clone());
5          }
6      }
7      for (local, typealt) in &function.locals {
8          symboltable.insert(&local.ident.string, typealt.clone());
9      }
10
```

Smaller checks over a big one

Rather than doing everything with one `analyse()` function which does checks at once, the work is modularly split into eight small check functions (besides the one checking for operand types which took half the file), each responsible for one kind of invariant. This kept the, in my opinion, most difficult part of the assignment a bit more manageable, and also made the testing straightforward, as it was simply a matter of testing every check.

Type checking statements

As previously mentioned, the largest stage in the semantic analyser is `check_statements`. For the majority of statements, the rule is simply that the type of the left-hand side must equal the type of the right hand side. The left-hand side type is derived from the lookup table for the local, while for the right side the type is derived from the operation performed.

A few cases deviate from this idea, and are handled separately; notably:

- `const`, for which the type of literal is checked to be compatible with the destination, rather than equal.
- `store`, which has no left-hand side at all, and which is instead checked for pointer and pointee type equality
- `member_ptr` for which:
 - 1) the operand must have type `ptr<P>`.
 - 2) The path P must be declared by an `extern type` block.
 - 3) The field must exist
 - 4) The result type is `ptr<T>`, where T is the field type.

Using the CFG

Three of the checks (label uniqueness, entry block exists, and that every jump target exists) are about control-flow integrity, so they run over the `Cfg` previously built. Whilst this wasn't strictly necessary, it helped simplify most of the logic there was no need to rederive structure

4.4 Error Handling

Like the parser, the pass stops at the first problem and reports it; for example, the error for a mismatched type for CJump's condition:

Cjump's condition type must be a boolean. Got <type> instead.

Limitation / known issue

One limitation that will be included in the later section regarding future improvements is location handling to easily find the errors for the semantic pass. Regardless, each check returns a descriptive message naming what went wrong, as shown above.

4.5 Testing

Testing Methodology

Semantic testing revolved around confirming that each of the proposed invariant checks worked, as well as that a valid program such as those in the assignment spec are not wrongly rejected

Test / Input	Expected outcome
Three assignment examples	Ok: well-formed
Undeclared local	Err: has no type
Duplicate block labels	Err: duplicate block label
Entry label with no block	Err: entry block does not exist
jump to missing label	Err: targets dont exist
cjump on a non-bool	Err: condition must be a boolean
Return type mismatch	Err: return type is incorrect
bin add on mismatched types	Err: expects matching numeric operands
load from a non-pointer	Err: load expects a pointer
store of the wrong type	Err: store cannot put incorrect type
member_ptr on a missing field	Err: field does not exist
Undeclared custom type	Err: referenced but not declared

5 Code Generation

5.1 Task Overview

The Code generator is the final process in the Compilation step. It takes as input a program which has passed the validation check, and returns a string, which then, in main, is written to a file. Ultimately, this step produces a compilable C file which implements the same function as the input ResIR code.

This process, like the semantic analyser lives in a single file:

- `generator.rs`: builds the C source piece by piece, and contains necessary functions to convert resir to C.

5.2 Design

Label-based generated code

The generator emits label-based C using `goto` rather than trying to rebuild structured `if` or `while` statements. This was, for me, the easier choice, especially since the idea of the control flow graph, had been burned into my mind by now. The control flow follows into basic steps: each block becomes a C label, and lowering the terminators is then almost mechanical. A `jump` becomes a `goto`, a `cjump` becomes an `if/else` over two `gotos`, and a `return` stays a `return`.

Conditional includes

Instead of emitting a fixed set of headers, the generator only adds the ones the program actually uses: `<stdint.h>` when a fixed-width integer appears, `<stdbool.h>` for `bool`, `<stddef.h>` when a `null` literal is generated (for C equivalent `NULL`) Additionally, the provided custom header is included when the program references a custom type or calls an external helper.

A standalone entry point

The assignment requires that the generated file compile with the exact command `gcc -std=c11 -Wall -Wextra -pedantic generated.c`. Since that also links an executable, a self-contained function needs a `main` to link on its own. The generator therefore appends a small stub `main` (unless the function is itself called `main`). To avoid this clashing with an external `main` supplied by a test driver, the stub is marked weak under GCC so that if another `main` is present at link time that takes over

Consuming the analysis

The generator also takes the list of unreachable blocks found by the Datalog stage (discussed in the next chapter), and simply skips them while emitting, so dead blocks never appear.

```
1   let unreachable_blocks = get_unreachables(&cfig);
2   if unreachable_blocks.is_empty() {
3       println!("Datalog analysis detected no unreachable blocks. Good job.\n");
4   } else {
5       println!("Datalog analysis eliminated {} unreachable block/s\n",
6           unreachable_blocks.len());
7   }
8   let c_code = generate_c(&program, header_name, &unreachable_blocks);
```

5.3 Main Functions

Function / Structure	Role
<code>generate_c(...)</code>	The main entry point. Takes as input the program, the header file name, and the unreachable blocks generated from the following step. Assembles the full C string: includes, signature, local declarations, entry block, and each block's label, statements, and terminator.
<code>type_to_c(type)</code>	For assisting with Type Mapping: it maps a ResIR type to its C equivalent.
<code>include_includes(...)</code>	Works out which headers the program actually needs and emits only those.
<code>pathsepto_(path)</code>	Joins a path's parts with underscores to form a C identifier ($A::B::C \rightarrow A_B_C$).
<code>convert_ops(stmt)</code>	For Operation lowering. Converts a ResIR operation to its C equivalent.
<code>convert_terminators(term)</code>	Lowers a terminator: <code>jump</code> $\rightarrow$ <code>goto</code> , <code>cjump</code> $\rightarrow$ <code>if/else</code> , <code>return</code> $\rightarrow$ <code>return</code> .

5.4 Type Mapping

ResIR type	C type
<code>i32</code>	<code>int32_t</code>
<code>i64</code>	<code>int64_t</code>
<code>u32</code>	<code>uint32_t</code>
<code>f64</code>	<code>double</code>
<code>bool</code>	<code>bool</code>
<code>ptr<T></code>	<code>T *</code>
<code>A::B::C</code>	<code>A_B_C</code>

5.5 Operation Lowering

ResIR operation	C lowering
<code>const</code>	assignment to the literal (<code>null</code> $\rightarrow$ <code>NULL</code>)
<code>bin (add, sub, ..., and, or)</code>	the matching C operator, e.g. <code>+</code> , <code>-</code> , <code>%</code> , <code>&&</code> ...
<code>un neg / un not</code>	<code>x = -a;</code> / <code>x = !a;</code>
<code>cast %a to T</code>	<code>x = (T)a;</code>
<code>addr_of</code>	<code>x = &a;</code>
<code>member_ptr</code>	<code>x = &p->field;</code>
<code>load</code>	<code>x = *p;</code>
<code>store</code>	<code>*p = v;</code>
<code>call</code>	ordinary C call, <code>::</code> $\rightarrow$ <code>_</code>

<code>jump bbX</code>	<code>goto bbX;</code>
<code>cjump %c, bbX, bbY</code>	<code>if (c) goto bbX; else goto bbY;</code>
<code>return [%x]</code>	<code>return x; or return;</code>

5.6 Testing

Testing Methodology

For Code Generation, there are no *direct* tests from the test harness; however, some tests regarding the program analysis section which follows this require functioning program generation, therefore I decided to join the two tests to reduce unnecessary automated testing. However, that being said, this part is still tested manually, as will be shown in the video; the generated C file is tested to be compileable using the appropriate command, and the code implements the specified function correctly.

Note

The requirement for the report to include three input ResIR functions together with the generated C will be fulfilled near the end of the report, in the appendix section.

6 Program Analysis with Ascent

6.1 Task Overview

The stage of program analysis involves a an analysis over the CFG built by the parser. The analysis I chose was unreachable-block detection, for similar reasons for choosing the label based lowering for code generation; familiarity with the idea of a Control Flow Graph. The analysis, after finding unreachable blocks of code(if any) then derives as set of such, which as shown beforehand, is consumable by the `generate_c` function from code generation, so to eliminate any unreachable blocks of code

6.2 Design

Ascent

Given the choice of implementation with Rust, Ascent, being an embedded Datalog library *for* Rust was the obvious choice.

Facts and rules

This is the clearest instance where having the CFG built from the parser task was a great time saver. The facts come straight from the CFG: Each CFG edge becomes an `edge` fact, and the entry block becomes an `entry` fact.

The Derived relation, `reachable`, is computed then by the two following rules:

- The entry block is **always** reachable
- Any block reachable from a reachable block, is automatically reachable.

```
1 // facts
2 relation edge(Label, Label);
3 relation entry(Label);
4
5 // derived
6 relation reachable(Label);
7
8 // rules
9 reachable(block) <-- entry(block);
10 reachable(to) <-- reachable(from), edge(from, to);
```

Producing the Set of Unreachable Blocks

Ascent then runs the rules above to a fixpoint, from which it then produces the set of reachable blocks. To compute the set of *unreachable* blocks, the function `get_unreachable(cfg)` walks the full CFG's blocks, and finds any blocks not in the set of reachable blocks; any one out is defined as unreachable.

Output Consumable by the compiler

As previously mentioned during the discussion on code generation; the output of the analysis; a set of unreachable blocks, is fed into the function which generates C so to eliminate any unreachable blocks during generation.

6.3 Testing

Testing Methodology

Testing the Program analysis involved confirming that the right blocks were confirmed to be unreachable / reachable, aswell as testing that the generated output eliminates the correct blocks.

Test / Input	Expected outcome
Block with no incoming edge	flagged unreachable (one block)
correct jump chain	nothing unreachable
cjump with both targets used	nothing unreachable
cjump plus a stray block	only the stray block unreachable
Dead block, then generate C	generated C omits that block's label
Reachable blocks, then generate C	generated C keeps bb0, bb1, bb2

7 Concluding Notes

7.1 Limitations

- Semantic Analysis Errors are currently not able to report location
- Although the tests were done with trying to cover my bases, it is realistically impossible to test for everything; this however implies that it is impossible to verify that it all works

7.2 Future Improvement

- Broaden test coverage as much as feasibly possible
- Eliminate any manual checking that happens for code generation by automating compilation of the C code
- Implement Structural based Code generation, potentially giving the user the option to choose between label based and structural based C
- Add error recovery, aswell as better more useful suggestions for the user
- Implementation of more than one type of program analysis, such as live-variable analysis which allows for removal of unused locals
- Due to this being the first project I have programmed in Rust, I am sure that the code can be **much** more performant and in areas possibly simplified

7.3 A Note on Rust for Compiler Construction

Rust was particularly suited to this task because of its performance, pattern matching, and strong type system. Its built-in testing support also made it easy to create a test harness alongside the implementation without needing extra tools. I appreciated that Rust provided some of the structure associated with object-oriented programming, through data types and `impl` blocks, without forcing the rigid class hierarchies used in languages such as Java. Features such as `match` were useful throughout the project, especially when handling tokens, IR nodes, statements, and types. The type system and borrow checker also made the implementation more reliable, and had a very high chance that if it compiled, it functioned as expected.[3]

7.4 Deviations from the Supplied grammar

No intentional deviations were made. The grammar was implemented as closely as possible to how it was as given.

Bibliography

- [1] Robert Nystrom. *Crafting Interpreters*. Genever Benning, 2021.
- [2] Thomas Reps. “maximal-munch” tokenization in linear time. *ACM Transactions on Programming Languages and Systems*, 20(2):259–273, 1998.
- [3] The Rust Project Developers. The rust programming language. <https://doc.rust-lang.org/book/>, 2024. Accessed: 2026-06-26.

A Sample ResIR Inputs

A.1 Assignment Example 1

```
1      function Math::abs_diff(%a: i32, %b: i32) -> i32 {
2          locals {
3              %zero    : i32;
4              %d        : i32;
5              %is_neg   : bool;
6              %r        : i32;
7          }
8          entry bb0;
9          bb0:
10         %zero    = const 0;
11         %d        = bin sub %a, %b;
12         %is_neg   = bin lt %d, %zero;
13         cjump %is_neg, bb1, bb2;
14
15         bb1:
16         %r = un neg %d;
17         return %r;
18
19         bb2:
20         return %d;
21     }
22
```

A.2 Assignment Example 2

```
1      extern type Custom::Struct::Point {
2          x : i32;
3          y : i32;
4      }
5      function Geom::point_x(%p: ptr<Custom::Struct::Point>) -> i32 {
6          locals {
7              %fp : ptr<i32>;
8              %v  : i32;
9          }
10         entry bb0;
11
12         bb0:
13         %fp = member_ptr %p, x;
14         %v = load %fp;
15         return %v;
16     }
17
```

A.3 Assignment Example 3

```
1      function Client::sum_abs_diffs(%a: i32, %b: i32,  
2      %c: i32, %d: i32) -> i32 {  
3          locals {  
4              %dx : i32;  
5              %dy : i32;  
6              %r  : i32;  
7          }  
8          entry bb0;  
9  
10         bb0:  
11         %dx = call Math::abs_diff(%a, %b);  
12         %dy = call Math::abs_diff(%c, %d);  
13         %r  = bin add %dx, %dy;  
14         return %r;  
15     }  
16
```

B Generated C Outputs

B.1 Assignment Example 1

```
1  #include <stdint.h>
2  #include <stdbool.h>
3
4  int32_t Math_abs_diff(int32_t a, int32_t b) {
5      int32_t zero;
6      int32_t d;
7      bool is_neg;
8      int32_t r;
9
10     goto bb0;
11
12     bb0:
13     zero = 0;
14     d = a - b;
15     is_neg = d < zero;
16     if (is_neg) goto bb1; else goto bb2;
17
18     bb1:
19     r = -d;
20     return r;
21
22     bb2:
23     return d;
24 }
25
26 #if defined(__GNUC__)
27 __attribute__((weak))
28 #endif
29 int main(void) {
30     return 0;
31 }
32
33
```

B.2 Assignment Example 2

```
1  #include <stdint.h>
2  #include "custom_header.h"
3
4  int32_t Geom_point_x(Custom_Struct_Point *p) {
5      int32_t *fp;
6      int32_t v;
7
8      goto bb0;
9
10     bb0:
11     fp = &p->x;
12     v = *fp;
```

```
13     return v;
14 }
15
16 #if defined(__GNUC__)
17 __attribute__((weak))
18 #endif
19 int main(void) {
20     return 0;
21 }
22
23
```

B.3 Assignment Example 3

```
1  #include <stdint.h>
2  #include "custom_header.h"
3
4  int32_t Client_sum_abs_diffs(int32_t a, int32_t b, int32_t c, int32_t d) {
5      int32_t dx;
6      int32_t dy;
7      int32_t r;
8
9      goto bb0;
10
11     bb0:
12     dx = Math_abs_diff(a, b);
13     dy = Math_abs_diff(c, d);
14     r = dx + dy;
15     return r;
16 }
17
18 #if defined(__GNUC__)
19 __attribute__((weak))
20 #endif
21 int main(void) {
22     return 0;
23 }
24
25
```

C Full Test Harness Output

```
1  running 28 tests
2  test test::binary_operation_pareses ... ok
3  test test::cjump_pareses ... ok
4  test test::const_assignment_pareses ... ok
5  test test::function_call_pareses ... ok
6  test test::lexer_returns_error_for_bad_characters ... ok
7  test test::member_ptr_pareses ... ok
8  test test::lexer_produces_correct_tokens ... ok
9  test test::missing_entry_doesnt_parse ... ok
10 test test::no_semicolon_doesnt_parse ... ok
11 test test::semantic_rejects_bad_bin_operands ... ok
12 test test::semantic_rejects_cjump_non_bool ... ok
13 test test::semantic_rejects_duplicate_labels ... ok
14 test test::semantic_rejects_load_from_non_pointer ... ok
15 test test::parser_passes_assignment_examples ... ok
16 test test::semantic_rejects_member_ptr_missing_field ... ok
17 test test::semantic_rejects_missing_entry_block ... ok
18 test test::semantic_passes_assignment_examples ... ok
19 test test::semantic_rejects_missing_jump_target ... ok
20 test test::semantic_rejects_store_wrong_value_type ... ok
21 test test::semantic_rejects_undeclared_custom_type ... ok
22 test test::semantic_rejects_return_type_mismatch ... ok
23 test test::task5_branch_all_reachable ... ok
24 test test::task5_keeps_reachable_blocks_in_generated_c ... ok
25 test test::semantic_rejects_undeclared_local ... ok
26 test test::task5_branch_with_dead_block ... ok
27 test test::task5_removed_block_is_not_in_generated_c ... ok
28 test test::task5_returns_empty_when_all_blocks_reachable ... ok
29 test test::task5_detects_unreachable_block ... ok
30
31 test result: ok. 28 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out;
32 finished in 0.00s
33
```

D output_checker.c used in video

```
1  #include <stdio.h>
2  #include <stdint.h>
3  #include "custom_header.h"
4
5  int main(void) {
6      int32_t r1a = Math_abs_diff(7, 3);
7      int32_t r1b = Math_abs_diff(3, 7);
8
9      int32_t r2 = Client_sum_abs_diffs(7, 3, 2, 9);
10
11     Custom_Struct_Point p;
12     p.x = 5;
13     p.y = 8;
14     int32_t r3 = Geom_point_x(&p);
15
16     printf("Example 1a: Math_abs_diff(7, 3)\n");
17     printf("Expected: 4\n");
18     printf("Got:      %d\n\n", r1a);
19
20     printf("Example 1b: Math_abs_diff(3, 7)\n");
21     printf("Expected: 4\n");
22     printf("Got:      %d\n\n", r1b);
23
24     printf("Example 2: Client_sum_abs_diffs(7, 3, 2, 9)\n");
25     printf("Expected: 11\n");
26     printf("Got:      %d\n\n", r2);
27
28     printf("Example 3: Geom_point_x({x = 5, y = 8})\n");
29     printf("Expected: 5\n");
30     printf("Got:      %d\n", r3);
31
32     return 0;
33 }
34
35
```